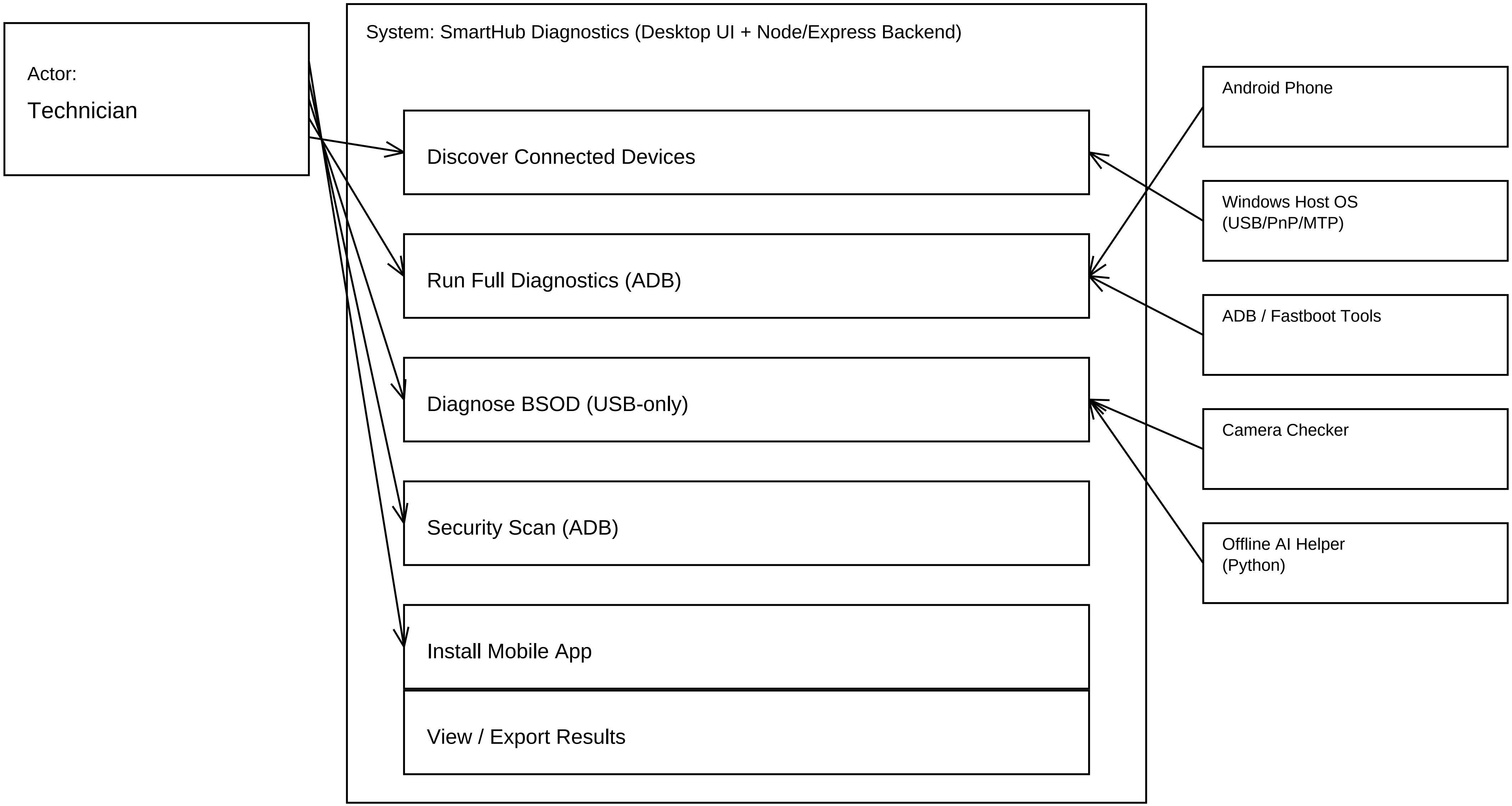
SmartHub Diagnostics â Use Case Diagram (Simplified)

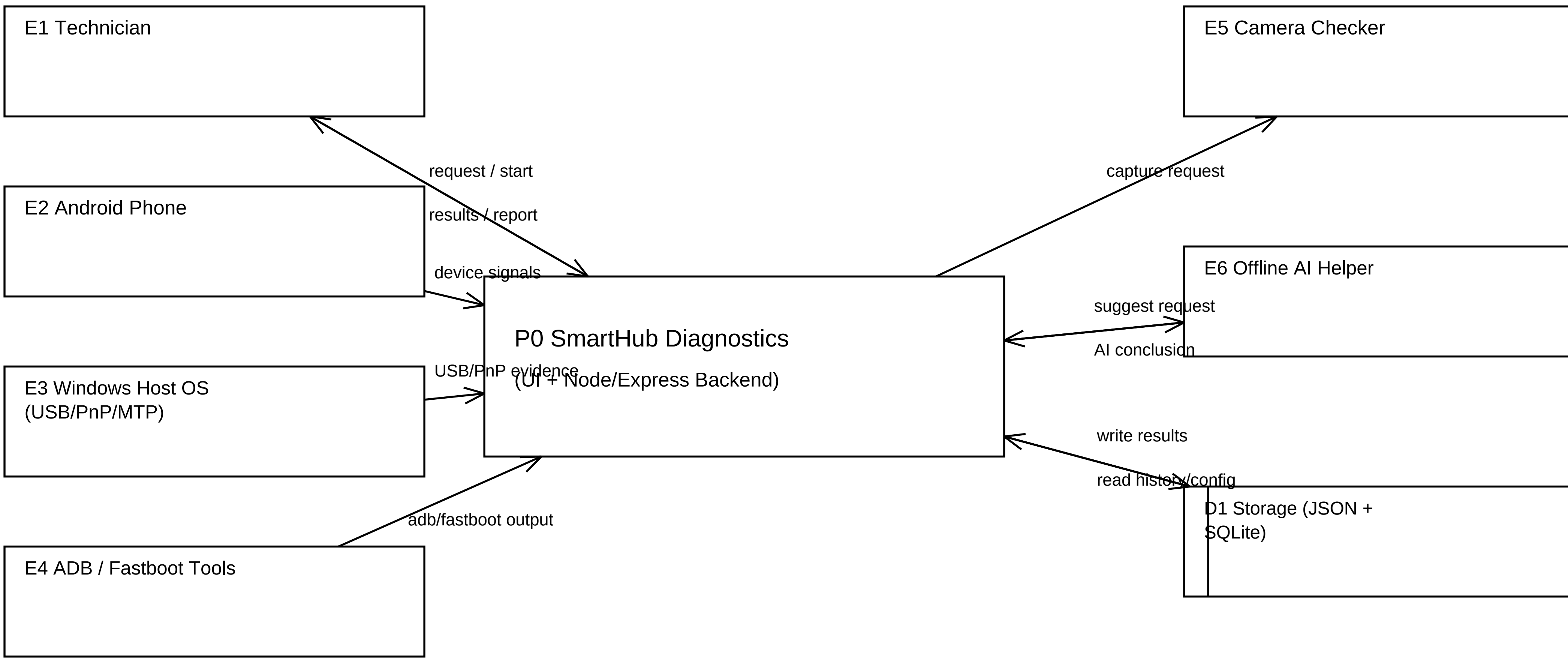


How to read (flow)

Technician starts a use case inside the SmartHub system boundary.
Boxes on the right are external systems used by SmartHub (phone, Windows USB/PnP, ADB/Fastboot tools, camera checker, and offline AI helper).
Arrows show interactions: Technician -> SmartHub use cases; SmartHub -> external systems when needed.
USB-only BSOD flow works without USB debugging (skipAdb/usbOnly); camera + offline AI are optional helpers.

SmartHub Diagnostics â DFD Level 0 (Context)

Method: Gane and Sarson DFD notation

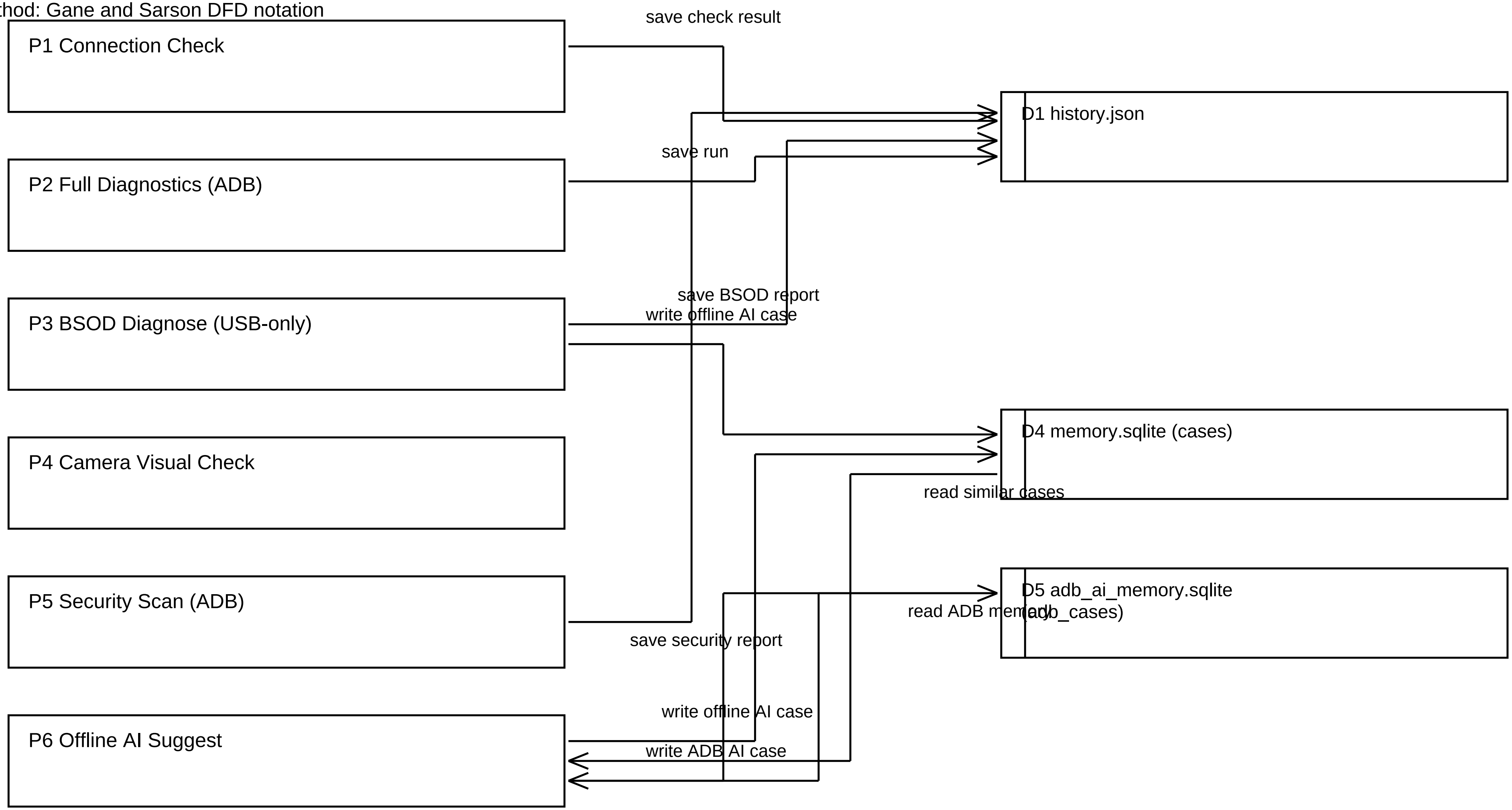


State / Define / Justify

State (In this level): SmartHub is represented as one process (P0) with only external entities and top-level flows.
Define (According to): Gane and Sarson define a context diagram as the whole system shown as one process with external interfaces (Gane & Sarson, 1979).
Justify (Moreover): This level gives a clear system scope before internal decomposition, which improves stakeholder understanding.

SmartHub Diagnostics â DFD Level 1 (Major Processes)

Method: Gane and Sarson DFD notation

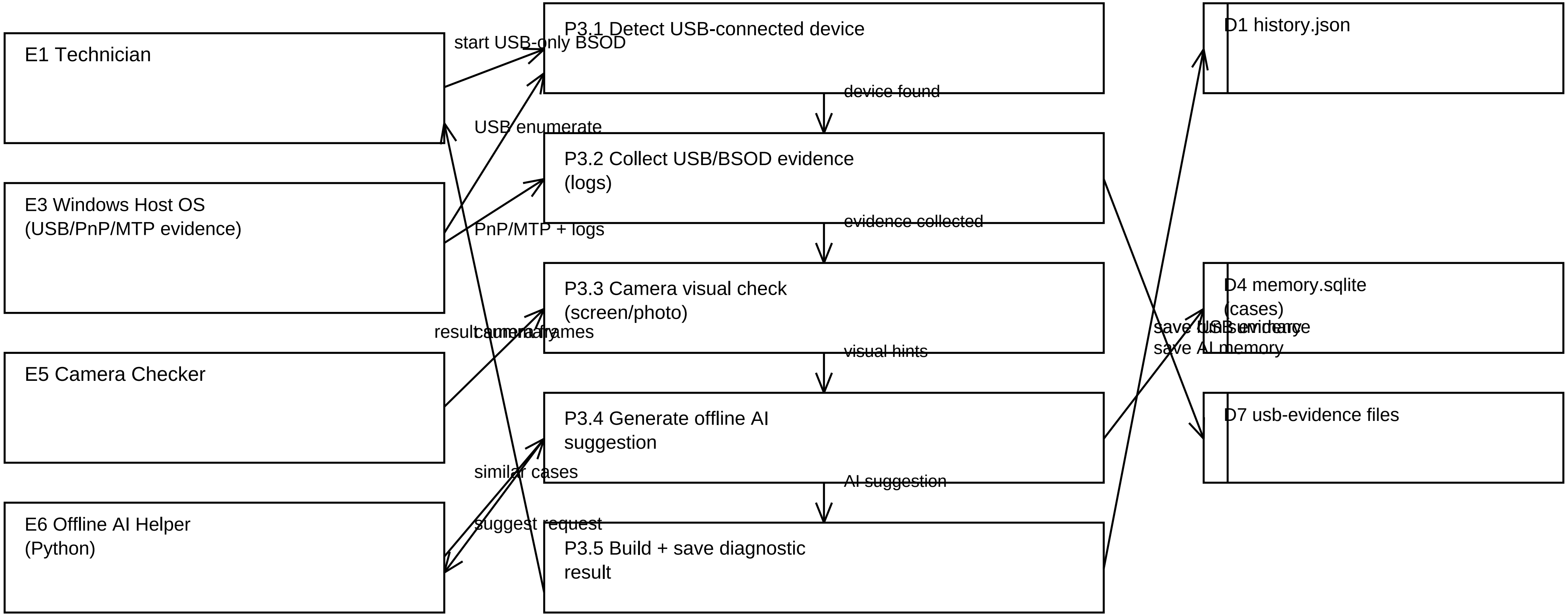


State / Define / Justify

State (In this level): P0 is decomposed into major SmartHub processes (P1-P6) and connected to persistent stores.
Define (According to): In Gane and Sarson leveling, the parent process is decomposed while preserving balanced input/output flows (Gane & Sarson, 1979).
Justify (Moreover): This level is needed to separate core functions such as connection check, ADB diagnostics, USB-only BSOD flow, and offline AI support.

SmartHub Diagnostics â DFD Level 2 (P3 BSOD Diagnose â USB-only)

Method: Gane and Sarson DFD notation



State / Define / Justify

State (In this level): P3 is decomposed into P3.1-P3.5 for detailed USB-only BSOD diagnosis flow.
Define (According to): Gane and Sarson Level 2 expands one Level 1 process into lower-level sub-processes while preserving parent-level data balance (Gane & Sarson, 1979).
Justify (Moreover): This decomposition is required because USB evidence handling, optional camera/AI support, and final reporting are distinct transformations.

SmartHub Diagnostics â ERD (Crow’s Foot Notation)

Storage: JSON/files (runs, screenshots, USB evidence) + SQLite (offline AI memory).

Technician

PK	technician_id INT
	username VARCHAR(50) UNIQUE
	password_hash VARCHAR(255)
	name VARCHAR(80)
	Stored: SQLite (recommended)

DiagnosticRun

PK	run_id VARCHAR(40)
FK	technician_id INT (logical)
	created_at INT (epoch ms)
	mode VARCHAR(16)
	device_primary VARCHAR(120)
	device_info VARCHAR(4000) (JSON)
	result_summary VARCHAR(1500)

Screenshot

PK	screenshot_id VARCHAR(40)
FK	run_id VARCHAR(40) (logical)
	file_path VARCHAR(260)
	taken_at INT (epoch ms)
	Stored: screenshots/*.png

BsodUsbOnlyReport

PK	report_id VARCHAR(40)
FK	run_id VARCHAR(40) (logical)
	bsod_category VARCHAR(40)
	confidence VARCHAR(10)
	primary_reason VARCHAR(800)
	usb_signals_json VARCHAR(4000) (JSON)
	host_evidence VARCHAR(2000) (paths)
	Stored: history.json + usb-evidence/*

Offline AI: cases

PK	id INTEGER
	created_at INTEGER
	last_seen INTEGER
	seen_count INTEGER
	source VARCHAR(32)
	device_primary VARCHAR(120)
	fingerprint VARCHAR(128)
	device_hints_json VARCHAR(4000)

ADB AI: adb_cases

PK	id INTEGER
	created_at INTEGER
	feature_hash VARCHAR(128)
	features_json VARCHAR(8000)
	label VARCHAR(120)
	confidence REAL
	failing_json VARCHAR(4000)
	actions_json VARCHAR(4000)

How to read (relationships)

This ERD uses Crow’s Foot notation to show relationships between entities used by SmartHub diagnostics.
Symbols: o = optional (0), | = one (1), crowfoot = many (*). Example: Technician |â â o< DiagnosticRun means one technician can have zero or many runs.
Data type/length notes: SQLite does not enforce VARCHAR lengths; lengths here are recommended for documentation.